

GB-TRACKER PRO

Guida all'ambiente di sviluppo

Windows 10/11 & macOS (Apple Silicon e Intel)

EDIZIONE GIUGNO 2026 · RGBDS v1.0.1 · PYTHON 3.14 · SAMEBOY v1.0.3

a cura di **Luigi Massari**

Progetto personale open source · Licenza MIT · github.com/luigismith/gb-tracker-pro

1 PANORAMICA: COSA SERVE

GB-Tracker Pro è un music tracker a 4 canali per Game Boy Color, scritto interamente in assembly SM83 e compilato con il toolchain open source **RGBDS**. L'intero ambiente di sviluppo è gratuito, leggero (meno di 100 MB in totale) e funziona identico su Windows e macOS. Questa guida ti porta da zero a una ROM compilata e funzionante in circa 15 minuti.

Componente	Versione (giu 2026)	Ruolo	Richiesto
RGBDS	v1.0.1 (1 gen 2026)	Assembler + linker + header fixer (rgbasm, rgbblink, rgbfix)	Sì
Python	3.8+ (consigliata 3.14.5)	Script di build e generatori di asset (font, note, waveform)	Sì
Git	qualsiasi recente	Clonare il repository e gestire le versioni	Sì
reportlab + Pillow	ultime da pip	Solo per rigenerare manuale PDF e screenshot	No
Emulatore	SameBoy v1.0.3 / BGB / mGBA 0.10.5	Eseguire e testare la ROM (serve accuratezza CGB)	Sì

NOTA

La compilazione della ROM richiede **solo Python standard + RGBDS**: nessuna dipendenza pip. I pacchetti reportlab e Pillow servono esclusivamente se vuoi rigenerare il manuale PDF o gli screenshot.

2 INSTALLAZIONE SU WINDOWS 10 / 11

WINDOWS

2.1 — Python e Git con winget

Apri un **Terminale** (Windows Terminal o PowerShell) e installa Python e Git con il package manager integrato di Windows:

```
winget install -e --id Python.Python.3.14
winget install -e --id Git.Git
```

ATTENZIONE

Se preferisci l'installer da **python.org**, spunta la casella **"Add python.exe to PATH"** nella prima schermata: senza, il comando python non verrà riconosciuto dal terminale.

2.2 — RGBDS v1.0.1

Scarica il pacchetto Windows a 64 bit (**rgbds-win64.zip**) dalla pagina release ufficiale:

```
# Browser:
https://github.com/gbdev/rgbds/releases/tag/v1.0.1
# oppure da terminale:
curl -LO
https://github.com/gbdev/rgbds/releases/download/v1.0.1/rgbds-win64.zip
```

Estrai dallo zip questi file nella cartella **tools/** del progetto (lo script di build li cerca lì per primo):

```
rgbasm.exe  rgbblink.exe  rgbfix.exe  rgbgfx.exe
libpng16.dll  zlib1.dll
```

In alternativa puoi estrarli in una cartella qualsiasi (es. `C:\rgbds`) e aggiungerla al `PATH` di sistema: `build.py` usa prima gli eseguibili in `tools/`, poi quelli nel `PATH`.

2.3 — Clona e compila

```
git clone https://github.com/luigismith/gb-tracker-pro.git
cd gb-tracker-pro
python build.py
```

Se tutto è a posto l'ultima riga sarà:

```
[OK] Built GB_TRACKER_PRO.gbc (32768 bytes)
```

2.4 — Dipendenze opzionali (manuale e screenshot)

```
python -m pip install reportlab pillow
```

3 INSTALLAZIONE SU MACOS

MACOS · APPLE SILICON & INTEL

3.1 — Homebrew

Tutto il toolchain si installa con **Homebrew**. Se non lo hai già, apri il Terminale ed esegui:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

L'installer di Homebrew scarica automaticamente anche i Command Line Tools di Xcode se mancano. Su Apple Silicon, al termine, segui le istruzioni a video per aggiungere `/opt/homebrew/bin` al `PATH`.

3.2 — RGBDS, Git e Python

```
brew install rgbds git python
# verifica: deve stampare "rgbasm v1.0.1"
rgbasm --version
```

SUGGERIMENTO

La formula Homebrew di RGBDS è mantenuta dagli stessi sviluppatori gbdev: `brew install rgbds` installa sempre l'ultima release stabile, già compilata per la tua architettura (arm64 o x86_64). Niente Gatekeeper, niente quarantena.

3.3 — Clona e compila

Su macOS non serve copiare nulla in `tools/`: lo script di `build` rileva automaticamente i binari RGBDS nel `PATH`.

```
git clone https://github.com/luigismith/gb-tracker-pro.git
cd gb-tracker-pro
python3 build.py
# atteso: [OK] Built GB_TRACKER_PRO.gbc (32768 bytes)
```

3.4 — Dipendenze opzionali (in virtual env)

Il Python di Homebrew è "externally managed" (PEP 668): `pip install` diretto viene bloccato. Usa un virtual environment dentro al progetto:

```
python3 -m venv .venv
source .venv/bin/activate
pip install reportlab pillow
```

3.5 — Emulatore: SameBoy

SameBoy è l'emulatore Game Boy più accurato in assoluto ed è nativo macOS — la scelta ideale:

```
brew install --cask sameboy
open GB_TRACKER_PRO.gbc -a SameBoy
```

4 VERIFICA DELL'INSTALLAZIONE

Checklist finale: esegui questi comandi dalla cartella del progetto e confronta l'output.

Comando	Output atteso
rgbasm --version	rgbasm v1.0.1
python --version (Win) / python3 --version (mac)	Python 3.x.y (almeno 3.8)
git --version	git version 2.x
python build.py	[OK] Built GB_TRACKER_PRO.gbc (32768 bytes)

Infine apri **GB_TRACKER_PRO.gbc** nell'emulatore: deve comparire il pattern editor con il titolo **GB-TRACKER PRO** e un brano vuoto. Premi **Select + Start** per aprire il menu Save/Load: lo **Slot 4** deve risultare SAVED — contiene la demo "Per Elisa". Selezionala con **A**: se senti la melodia, l'ambiente è perfettamente funzionante.

Emulatore	Piattaforma	Note
SameBoy v1.0.3	macOS (nativo), Windows, SDL	Il più accurato; debugger integrato
BGB	Windows	Debugger eccellente, ottimo per sviluppo assembly
mGBA 0.10.5	Windows, macOS, Linux	Veloce e multiplatforma
binjgb	Browser (WebAssembly)	Zero installazione, utile per test rapidi

NOTA

Per provare la ROM su **hardware reale** basta copiare **GB_TRACKER_PRO.gbc** sulla microSD di una flashcart MBC5 (EZ-Flash Junior, Everdrive GB, EICheapoSD). I salvataggi finiscono in un file **.sav** da 32 KB gestibile con **tools/fts_tool.py**.

5 STRUTTURA DEL PROGETTO

```

gb-tracker-pro/
|-- build.py          build completo in un comando
|-- src/             sorgenti assembly SM83
|   |-- main.asm     boot, main loop, VBlank ISR
|   |-- player.asm   motore audio + demo song
|   |-- ui.asm       rendering pattern editor e menu
|   |-- input.asm    joypad e dispatch comandi
|   |-- save.asm     salvataggi SRAM (4 slot)
|   +-- font/notes/waves.asm (auto-generati, non editare)
|-- inc/hardware.inc  registri hardware GBC
|-- tools/           generatori Python + RGBDS (Windows)
+-- manual/          screenshot per manuale e README

```

ATTENZIONE

I file `font.asm`, `notes.asm` e `waves.asm` vengono **rigenerati a ogni build** dagli script in `tools/`: qualsiasi modifica manuale viene sovrascritta. Modifica invece i generatori.

6 RISOLUZIONE DEI PROBLEMI

Sintomo	Causa	Soluzione
rgbasn not found	RGBDS non installato o fuori PATH	Win: estrai gli exe in tools/. mac: brew install rgbds
Errore DLL all'avvio di rgbgfx (Win)	Mancano le librerie a fianco degli exe	Copia anche libpng16.dll e zlib1.dll dallo zip in tools/
python non riconosciuto (win)	PATH non aggiornato dall'installer	Reinstalla spuntando "Add to PATH", oppure usa il launcher: py build.py
externally-managed-environment (mac)	PEP 668: il Python di Homebrew blocca pip globale	Usa il venv come in §3.4
"App non verificata" (mac)	Gatekeeper sui binari scaricati a mano	xattr -dr com.apple.quarantine — oppure usa brew, che non ha questo problema
La ROM compila ma il manuale no	reportlab / Pillow non installati	pip install reportlab pillow (sono opzionali, la ROM non li richiede)
Lo slot 4 non contiene la demo	SRAM non ancora formattata dall'emulatore	Chiudi e riapri la ROM: al primo boot la SRAM viene formattata e la demo installata

7 RISORSE UTILI

Risorsa	URL
Repository del progetto	github.com/luigismith/gb-tracker-pro
RGBDS — documentazione ufficiale	rgbds.gbdev.io
Pan Docs — riferimento hardware GB/GBC	gbdev.io/pandocs
Community gbdev (tutorial, esempi, Discord)	gbdev.io
SameBoy	sameboy.github.io

Risorsa	URL
BGB	bgb.bircd.org
mGBA	mgba.io

Buon divertimento, e buona musica a 8 bit!

Luigi Massari

Giugno 2026 · progetto personale, rilasciato con licenza MIT